

研報向量搜索優化報告

混合檢索 (語意 + 關鍵字) 優化成果與效能驗證 — 新手版

日期：2026-06-11 系統：report-mark 研究報告檢索系統 (<http://localhost:8097>) 對象讀者：不熟悉向量搜索的使用者

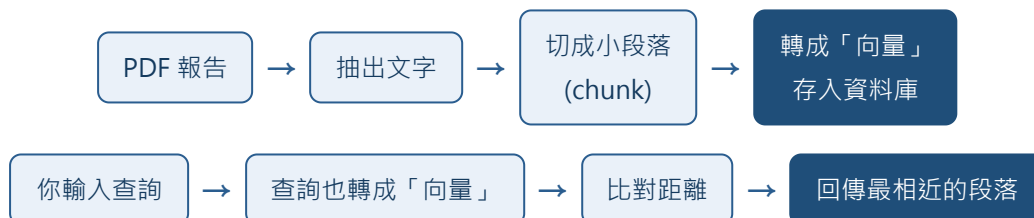
一、一句話總結

這次優化讓搜尋**又準又快**：搜「AI伺服器」會優先找到真正討論「AI伺服器」的報告 (而不是只講「AI」或只講「伺服器」的) ；重複查詢從約 **0.18 秒** 降到 **0.038 秒** 。另外，過去伺服器重啟後第一次查詢要等近一分鐘 (模型初始化) ，現在已改為啟動時自動暖機 (暖機 = 預先把模型載入記憶體準備好) ，使用者不會再碰到這個等待。

二、背景：這個系統在做什麼？

你有上萬份券商研究報告 (PDF) ，想用「語意」搜尋它們——例如輸入「記憶體漲價」，希望找到討論 DRAM 報價上漲的報告，即使報告裡寫的是「DRAM 合約價調漲」這種不同的措辭。

系統的運作流程：



目前資料庫裡有 **241 篇報告**、**10,295 個段落**；另有約 **1.4 萬篇**陸續導入中 (其中 7,167 篇已完成分類、隨時會進來，7,332 篇還在分類) 。

三、先弄懂三個基本概念

1. 什麼是「向量」 (embedding · 嵌入) ？

把一段文字交給 AI 模型 (我們用的是 **BGE-M3**) ，它會輸出一串 **1,024 個數字**——可以想像成這段文字在一個「語意地圖」上的座標。

關鍵特性：**意思相近的文字，座標就靠近**。

- 「DRAM 報價上漲」和「記憶體漲價」→ 座標很近
- 「DRAM 報價上漲」和「午餐吃什麼」→ 座標很遠

所以「找語意相近的文件」就變成「找座標距離最近的點」，這就是向量搜索。

2. 什麼是 HNSW 索引？

資料庫裡有一萬多個段落的座標，要找「離查詢最近的點」，最笨的方法是一個一個算距離（全部掃一遍）。**HNSW** 是一種「捷徑網絡」：像高速公路系統，先走快速道路跳到大概的區域，再走小路精確定位，不必掃完全部資料。資料量越大，省的時間越多。

3. 純向量搜索的天生弱點

向量搜索看的是「整體語意」，**不保證關鍵字一定出現**。這就是優化前的核心問題：

搜「AI伺服器」，模型把它理解成「跟 AI 和伺服器有關的東西」，結果只討論「AI 晶片」的報告、只討論「一般伺服器」的報告也會被排到前面，真正逐字寫到「AI伺服器」的報告反而沒被優先。

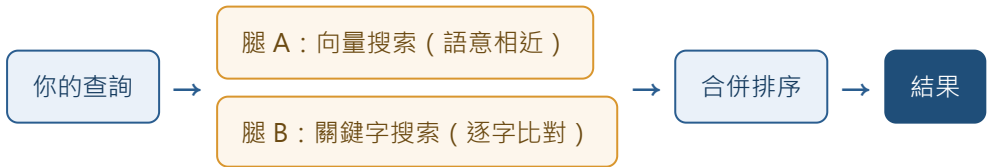
四、優化前有哪四個問題？

#	問題	白話說明
1	複合詞被拆散	搜「AI伺服器」回來一堆只講「AI」或只講「伺服器」的結果
2	隱藏的截斷 bug	程式要求資料庫回 60 筆候選，但 HNSW 的預設參數 (ef_search=40) 讓它最多只回 40 筆， 默默少了三分之一的候選 ，而且不會報錯
3	重複查詢一樣慢	每次查詢都要重新用 AI 模型算一次向量 (CPU 上要 0.1 ~ 0.3 秒；剛開機、模型還沒載入時更久)，就算你查的是同一個詞
4	PDF 文字有「怪空格」	PDF 抽出來的文字常變成「A I 伺 服 器」這種逐字斷開的形式，導致關鍵字比對永遠對不上

五、做了哪些優化？

優化 1：混合檢索 (Hybrid Search) ——這是主菜

從「只用向量」改成「**兩條腿走路**」：



- **腿 A (語意腿)**：原本的向量搜索，負責找出「意思相近但措辭不同」的段落
- **腿 B (關鍵字腿)**：新增的逐字比對，用 PostgreSQL 的 **pg_trgm** (三字組索引) 在全部段落裡快速找出「真的含有這幾個字」的段落

合併時採**三層優先排序** (tier)，層級高的永遠排在層級低的前面：

第一層 段落裡完整出現查詢詞組（如「AI伺服器」五個字連在一起）

第二層 查詢的每個詞都出現（「AI」和「伺服器」都有，但沒連在一起）

第三層 其餘的——查詢詞只出現一部分（按出現比例小幅加分），或純粹語意相近

這個分層是程式裡的硬性規則（先比層級、同層才比分數），保證逐字命中的結果永遠排在純語意聯想的結果前面，不靠運氣。

優化 2：文字正規化

新增一個自動同步的欄位 `content_norm`：把每個段落的文字「去掉所有空格、統一大小寫、統一全形半形」之後存起來。「AI伺服器」正規化後變成「ai伺服器」，關鍵字比對就對得上。已對既有 9,793 個段落做了一次性清理。

優化 3：查詢向量快取（LRU Cache）

同一個查詢字串的向量算一次就記住（最多記 256 個）。你在網頁上切換市場篩選、或重複搜同個詞時，不必重算向量，直接用現成的。

優化 4：修掉截斷 bug

舊版向資料庫要 60 筆候選、實際只拿到 40 筆（被預設參數默默截斷）。新版把候選池擴大到 **120 筆**，並把搜索廣度參數（`ef_search`）同步設到 120——廣度永遠不小於要求的筆數，截斷就不會再發生。候選變多，排序品質的「原料」也更充足。

六、實測成效（2026-06-11 驗證）

速度

測試場景	優化前	優化後實測	說明
重複查詢（已查過的詞）	約 0.18 秒（每次重算）	0.038 秒	快取直接命中
第一次查新的詞	0.1–0.3 秒起跳；重啟後首查曾測到近一分鐘	0.21–0.34 秒	主要花在算查詢向量；初始化已移到啟動暖機
帶市場篩選的查詢	—	0.17 秒	篩選不拖慢速度
資料庫端：關鍵字腿	不存在此功能	1.1 毫秒	trigram 索引生效
資料庫端：向量腿	候選被截斷至 40 筆	23 毫秒 ，完整 120 筆候選	詳見下方備註

備註——關於向量腿的 23 毫秒：

- 目前一萬個段落的規模下，資料庫一律選擇「全部掃一遍」（23 毫秒）而不走 HNSW 捷徑。這是資料庫自己算過的理性選擇：資料還不多，掃全部不吃虧，而且**一筆不漏（召回率 100%）**。
- HNSW 捷徑實測：剛啟動第一次 84 毫秒（資料還沒進記憶體），之後 1.4 毫秒。
- 等段落數成長到一定規模（推估三萬以上，未實測），資料庫會自動改走 HNSW，不需人工調整。
- 無論哪種方式都低於 30 毫秒的設計目標，使用上無感。

準確度（品質）

測試一：搜「AI伺服器」

	優化前	優化後
第一名分數	0.617	0.867
前五名分數帶	分散	0.793–0.867
詞組命中	結果混雜「只講 AI」「只講伺服器」的報告	第 2–4 名 每一個 回傳片段都逐字含有「AI伺服器」（第 1 名為 1/3 個片段命中）

注意：兩欄的計分方式不同——優化後的分數含逐字命中加分（完整詞組 +0.25），所以 0.617 → 0.867 不能直接解讀成「相似度提升 0.25」；真正的改善是**排序品質**：逐字命中的報告現在穩定排在前面。

測試二：搜「記憶體漲價」+ 篩選台股（TW）

第一名：南亞科（2408）——台灣的 DRAM 製造商，3/3 個片段全部逐字命中「記憶體」。這正是理想答案。

分數怎麼讀：1.0 是滿分（完全相同）。優化後的分數含「逐字命中加分」（完整詞組 +0.25、全詞 +0.15），所以 0.85 以上通常代表「語意相近**且**逐字命中」的高可信結果。

七、對你和同事的實際意義

1. **搜複合詞變可靠了**：「AI伺服器」「先進製程」「庫存調整」這類產業術語，現在會優先回傳真正討論該主題的報告。
2. **介面操作變順了**：切換市場/類型篩選、重送同樣的查詢，幾乎瞬間回應（0.04 秒級）。
3. **第一次查新詞仍需約 0.2–0.3 秒**：這是 AI 模型在 CPU 上算向量的固有成本，屬正常範圍。
4. **伺服器重啟後約 1 分鐘內暫時無法服務**：模型載入記憶體需要約 1 分鐘，載入完成後即恢復正常。因為啟動流程會自動暖機，一般使用者不會碰到「第一次查詢特別慢」的情況。

八、目前限制與下一步

- **語料還在擴充**：7,167 篇已分類報告正在背景導入（CPU 嵌入是瓶頸，預計需數天），另有 7,332 篇待分類。全部完成後語料會從 241 篇成長到約 1.5 萬篇規模。
- **向量品質與文字清理的取捨**：既有段落的向量是用「清理前」的文字算的，這次只清理了文字本身、沒有重算向量（成本太高）。實測影響可忽略，新導入的文件則一律先清理再算向量。
- **規模成長後的自動切換**：HNSW 索引基礎設施已就緒，資料量變大時資料庫會自動改走捷徑，不需人工調整。

附錄：名詞小辭典

名詞	白話解釋
向量 / 嵌入 (embedding)	文字的「語意座標」，一串 1,024 個數字；意思越近、座標越近
BGE-M3	我們用來把文字轉成向量的 AI 模型，中英文都支援
chunk (段落塊)	報告切成的小段文字，是搜尋與比對的最小單位
pgvector	PostgreSQL 資料庫的向量擴充，讓資料庫能存向量、算距離
HNSW	向量的「捷徑網絡」索引，大資料量時免掃全部就能找到最近的點
ef_search	HNSW 搜索時探索的廣度；太小會默默漏掉候選（就是問題 2 的元兇）
pg_trgm	把文字拆成連續三個字的小片段做索引，讓「包含某關鍵字」的查詢飛快
LRU 快取	「最近用過的留著、最久沒用的丟掉」的記憶機制，用來避免重算查詢向量
召回 (recall)	該找到的有沒有都找到；截斷 bug 傷的就是召回
tier (層級排序)	本次設計的排序規則：完整詞組 > 全詞命中 > 部分命中/純語意相近